

UNIT-I

UNIT - I

Introduction:

Agile Methods

Agile Manifesto, and Agile Modelling

What Is Agile ?

XP: Extreme Programming

DSDM, SCRUM

Feature-Driven Development

Modelling Misconceptions

Agile Modelling

Tools of Misconceptions

Updating Agile Models.

UNIT-I

- Introduction:

In this UNIT, you will be introduced to the concept, motivations, goals and Principles behind the Agile Movement. You will also learn what are the common features of all agile methods are, a brief review on the number of the core agile methods currently available.

What Is Agile?

Agile *adj. nimble, quick-moving*

- That is, they aim to be nimble and quick moving in response to changes in requirements, to the people that comprise the development teams and to issues that arise during the software development process.
- So agile methods are methods that try to focus on the primary goal of software development, i.e., the creation of working (defect-free) software
- Agile also implies being both effective and sufficient for the current situation. By this I mean:
 - *effective in terms of producing working, defect-free (as far as possible) software,*
 - *sufficient in terms of meeting its requirements both in the short-term and in the long-term (for example, the documentation required to support a long-lived project will be greater than for a short-lived one).*

what is an *Agile Methodology*?

It is a method that tries to be responsive to the needs of the software development process, that is based on practise and experience, and that focuses on being effective and sufficient!

Agile methods

- ✧ Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods.
- ✧ Agile development is a philosophy; It's a way of thinking about software development
- ✧ Agile methods are processes that support the agile philosophy.
 - Examples include Extreme Programming and Scrum.

Agile methods – Contd.

- ✧ Agile methods consist of individual elements called practices
 - Agile methods combine them in unique ways, accentuating those parts that support the agile philosophy, discarding the rest, and mixing in a few new ideas

Agile methods – Contd.

✧ Agile methods:

- Focus on the code rather than the design
- Are based on an iterative approach to software development
- Are intended to deliver working software quickly and evolve this quickly to meet changing requirements

✧ The aim of agile methods is

- To reduce overheads in the software process (e.g. by limiting documentation) and
- To be able to respond quickly to changing requirements without excessive rework

Agile Manifesto

Agile Software Development Alliance produced a manifesto that they hoped embraced the philosophies that they commonly supported, and that they believed to help to produce better software. From this manifesto, they defined a set of principles for Agile Software Development.

The manifesto proposed the following values (very briefly) summarised below:

Agile Manifesto

- ✧ The description of the agile philosophy is the Agile Manifesto, a collection of 4 values and 12 principles
- ✧ We are uncovering better ways of developing software by doing it and helping others do it.
- ✧ Through this work we have come to value:
 - Individuals and interactions over processes and tools
 - Working software over comprehensive documentation
 - Customer collaboration over contract negotiation
 - Responding to change over following a plan
- ✧ That is, while there is value in the items on the right, we value the items on the left more.

1. Individuals and interactions over processes and tools.

This refers to the fact that it is the people involved and how they communicate that typically has the largest bearing on the success (or failure) of a software project. Yes, software development processes, methodologies, tools, etc., can help but they are still not the overriding influence. Thus, you should encourage the best people and group interactions.

2) Working software over comprehensive documentation.

At the end of the day it is the software produced by a development project that will be used by a user and not the documentation. Therefore, documentation should not be a major goal in and of itself. Instead, it should be a supporting medium for the actual product – the software.

3. Customer collaboration over contract negotiation.

Remember these are values – thus, time should be spent on working with customers and in getting them involved in the software development rather than on detailed contract negotiations

For example, we have worked with a number of clients where the legal (and/or financial) departments have imposed a contract and associated negotiations because they want something to hit us with, if things go wrong.

4. Responding to change over following a plan

Finally, agile software development embraces change rather than saying “it’s not in the requirements or the plan, so we can’t do it.” That is development progresses in response to user feedback, rather than as a reaction to a fixed plan.

4 Key Agile Values

**Individuals &
Interactions**
over processes
and tools



**Customer
Collaboration**
over contract
negotiation



**Working
Software**
over comprehensive
documentation



**Responding
to Change**
over following a
plan



Agile Principles

Principles	Description
Customer Satisfaction	Our highest priority is to satisfy the customer through early and continuous delivery of valuable software
Changing Requirements	Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage
Deliver Frequently	Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale
Customer Collaboration	Business people and developers must work together daily throughout the project
Trust	Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done
Sitting Together	The most efficient and effective method of conveying information to and within a development team is face-to-face conversation

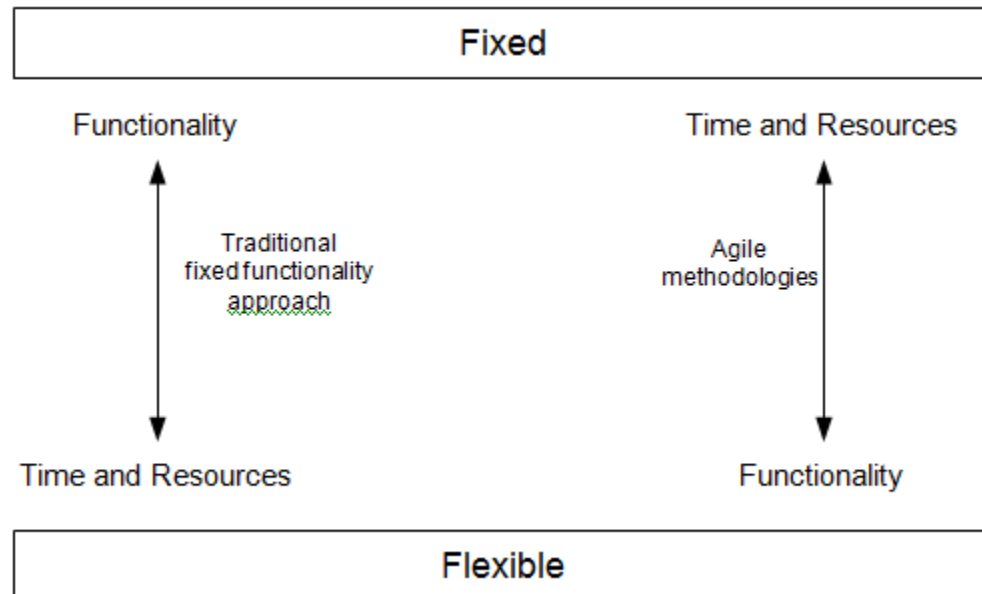
Agile Principles

Principles	Description
Working Software	Working software is the primary measure of progress
Sustainable Development	Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely
Technical Excellence	Continuous attention to technical excellence and good design enhances agility
Simplicity	Simplicity - the art of maximizing the amount of work not done - is essential
Self-Organizing Teams	The best architectures, requirements, and designs come from self-organizing teams
Adaptive Planning	At regular intervals, the team reflects on how to become more effective, then tunes and adjust its behavior accordingly

What Are Agile Methods?

For example, consider the following diagram.

This diagram tries to illustrate the difference in emphasis of agile methods compared to more traditional methods. That is, in an agile method it is the time available and the resources available that are generally considered to be fixed, while the functionality is considered more flexible. Thus, the aim is usually to set a fixed delivery date at which point something will be delivered and to prioritize the functionality that must be implemented so that what can be implemented will be implemented, but to acknowledge that not everything may be delivered.



Agile Modelling

Agile Modelling applies the philosophy of the agile movement to the software modelling process. It tries to find an appropriate balance between too little modelling, and too much modelling, at the design stage. That is, the point at which you have modeled enough to explore and document your system effectively, but not so much that it becomes a burden .

It is certainly not a methodology in its own right and rather more of an approach or philosophy towards the modelling stage of a project.

it has three main goals:

- The definition and promotion of a set of **values**, **principles** and **practices** that help to produce the appropriate models.
- Guidance on how to apply modelling to an agile software development.
- Guidance on how to apply agile modelling to other software processes (such as RUP).

This is defined in terms of a set of criteria that an agile model should meet, these are:

1. *Agile models provide positive value.*
2. *Agile models fulfill their purpose (and no more)..*
3. *Agile models are sufficiently accurate.*
4. *Agile models are sufficiently consistent.*
5. *Agile models are sufficiently detailed.*

XP: eXtreme Programming

- Extreme Programming, or as it is more commonly known XP, was originally designed as a way of supporting small development teams working within uncertain and changing requirements
- XP was designed as an approach based on software engineering principles, but focused on the timely delivery of software that meets users' requirements (rather than on the sometimes over bearing processes that surround the development of software).
- XP is the empowerment of the actual developers – they should be able to react immediately to changing customer requirements, even late in the development life cycle.
- XP also places great emphasis on the software development team and teamwork. The team, in turn, incorporates management, technical personnel and end users all cooperating towards the common good.
- It takes as one of its aims that teams communicate and constantly pay attention to all the details necessary to make sure that the software being developed matches the user requirements, to help to produce quality software.

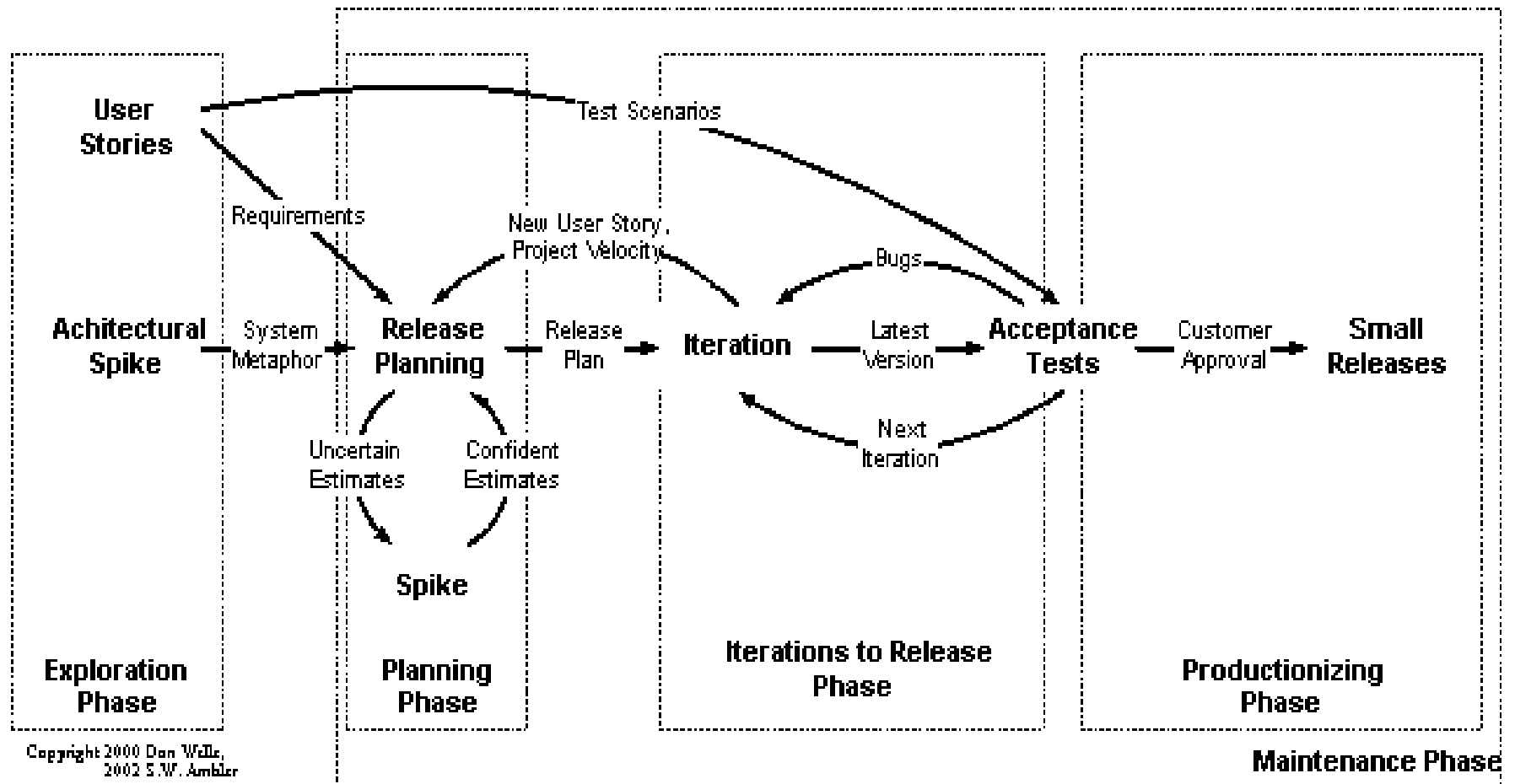
Underlying XP are four basic principles, these are:

- *Communication* – it is good to talk (particularly, between users and developers).
- *Simplicity* – keep it simple and grow the system as and when required.
- *Feedback* – let users provide feedback early and often.
- *Courage* – to go with such an approach.

These four basic principles have led to the following key ideas presented within XP:

- *Code in pairs.* This is probably the thing that people first hear with relation to XP. The idea is that all software is developed in pairs (i.e., with two programmers at one screen). The concept is that if code reviews are good, because they force at least one other person to consider your code, then constantly working in pairs results in constant reviewing of code and feedback between the two developers. *Stay in contact with the customer.* For example, place a customer representative in the team, so that you have access to them all of the time. Meet regularly with the customer to give information and receive feedback.
- *Create tests before coding then test heavily.* Developers should write the unit tests before the code to be tested. Part of the argument is that if you can't write the test (i.e., don't know what the inputs and outputs should be), then you shouldn't be writing the code. You should then automate testing so that you can regularly re-run the tests to make sure that nothing that has been done breaks earlier results.
- *Short iterations.* Each iteration should be relatively short allowing for rapid and frequent feedback. Thus, a minimal system may be produced and possibly even put into production quickly and the system will grow in whatever directions prove most valuable.
- *Keep it simple.* Start projects with a simple design that can evolve later as required by future iterations. This removes unnecessary complexity from early iterations. It also removes the need to code in additional functionalities believed to be required by future iterations, but which may actually never be needed.
- *Don't anticipate: code for current needs.* That is, don't over-engineer solutions based on what they may one day need to do, rather focus on what they need to do now and leave tomorrow's functionality to tomorrow.
- *Collective ownership.* Everyone within the team owns the software and has responsibility for it. When something goes wrong, no one should ever consider it not to be his or her problem because Bill wrote that piece of code. In turn, XP does not support a culture of blame and recrimination – everyone is responsible for all the code. As a result of this, everyone is responsible for fixing a problem when they find it rather than ignoring it.

The XP Project Lifecycle



Architectural Spike

A Spike in XP terms is an attempt to reduce the risk associated with an unknown area of the system, technology or application domain. A Spike may involve some investigation, research and possibly some software to evaluate or elucidate the problem. The result of most explorations is usually not good enough to keep and so should be thrown away. Early on the overall architecture of the system to be developed is an important issue that needs to be resolved. Thus, at this stage research and analysis of the architecture to use should be carried out and fed into the release planning meeting. Other spikes are used during the project-planning phase to determine unresolved issues.

Release Planning

- A release-planning meeting is used to create the release plan, which lays out the overall project. That is, the release plan indicates which user stories will be implemented and in which release this will happen. It also indicates how many iterations are planned and when each iteration will be delivered. This is done by negotiation between the interested parties using estimates derived from the user stories. The estimates are produced by the technical members of the team potentially with input from the users.

Iterations

- Each iteration within the project adds to the agility of the development team. That is, at the start of each iteration changes can be made to what will be done and when it will be done.
- The shorter the iteration, the quicker the development team can respond to changes (indeed within XP it is recommended that iterations last only a matter of weeks).
- At the beginning of each iteration, an Iteration Planning meeting is held to determine exactly what will happen within that iteration (such as what programming tasks there will be and when they will be done).
- It looks like the iteration will not manage to achieve all that was planned for it, then further iteration planning meetings should be called during the iteration to respond to these concerns

Acceptance Testing

The acceptance tests are created from the user stories, written at the start of the project. Each iteration implements one or more user stories; these stories will be translated into a series of acceptance tests during the iteration. To do this, the customer must specify scenarios to test whether a user story has been correctly implemented. A story is not limited to one acceptance test and may be related to many acceptance tests (depending on the complexity of the scenario).

Release

XP promotes the concept of “small releases” and “release often.” The release- planning meeting should identify meaning “chunks” of system functionality that makes business sense in relation to the users and to the state of the system at various intervals. These meaningful releases should be made available to users when available. This will allow early and frequent feedback from the users rather than relying on the big bang approach and then worrying about the consequences.

DSDM

- DSDM or The Dynamic Systems Development Method provides a framework of controls and best practice for Rapid Application Development (RAD). It is particularly suitable for application development projects that need to develop complex business solutions within tight timeframes.
- A worldwide consortium of systems developers initially designed (and indeed are still evolving) DSDM (now in Version 4.1). Their goal was to produce what at the time they referred to as a RAD methodology which has evolved into an Agile Method model that is time, quality and cost sensitive, producing deliverables quickly and accurately – rapid and right. Since its inception in 1995, more than 20,000 practitioners have been trained and thousands of developers have used DSDM successfully.
- A central tenant of DSDM is that “high-quality demands fitness for purpose as well technical robustness” rather than the need to match every requirement as described in the requirements document to the *n*th degree (not least because many requirements documents are at best flawed).

DSDM is based on nine overriding principles, these are:

- Active user involvement is imperative.
- The team must be empowered to make decisions.
- The focus is on frequent delivery of products.
- Fitness for business purpose is the essential criterion for acceptance of deliverables.
- Iterative and incremental development is necessary to converge on an accurate business solution.
- All changes during development are reversible.
- Requirements are base lined at a high level.
- Testing is integrated throughout the life cycle.
- Collaboration and cooperation between all stakeholders is essential.

The actual DSDM lifecycle is broken down into seven different phases, these are:

Pre-Project Phase, Feasibility Study, Business Case Study, the Functional Model Iteration (FMI), the Design and Build Iteration (DBI), the Implementation Phase and the Post-Project Phase.

These are illustrated in Figure ,The first three phases (namely, the Pre-Project, Feasibility and Business Studies phases) are done sequentially in order. These phases set the ground rules for the rest of development process allowing users and teams to understand the world

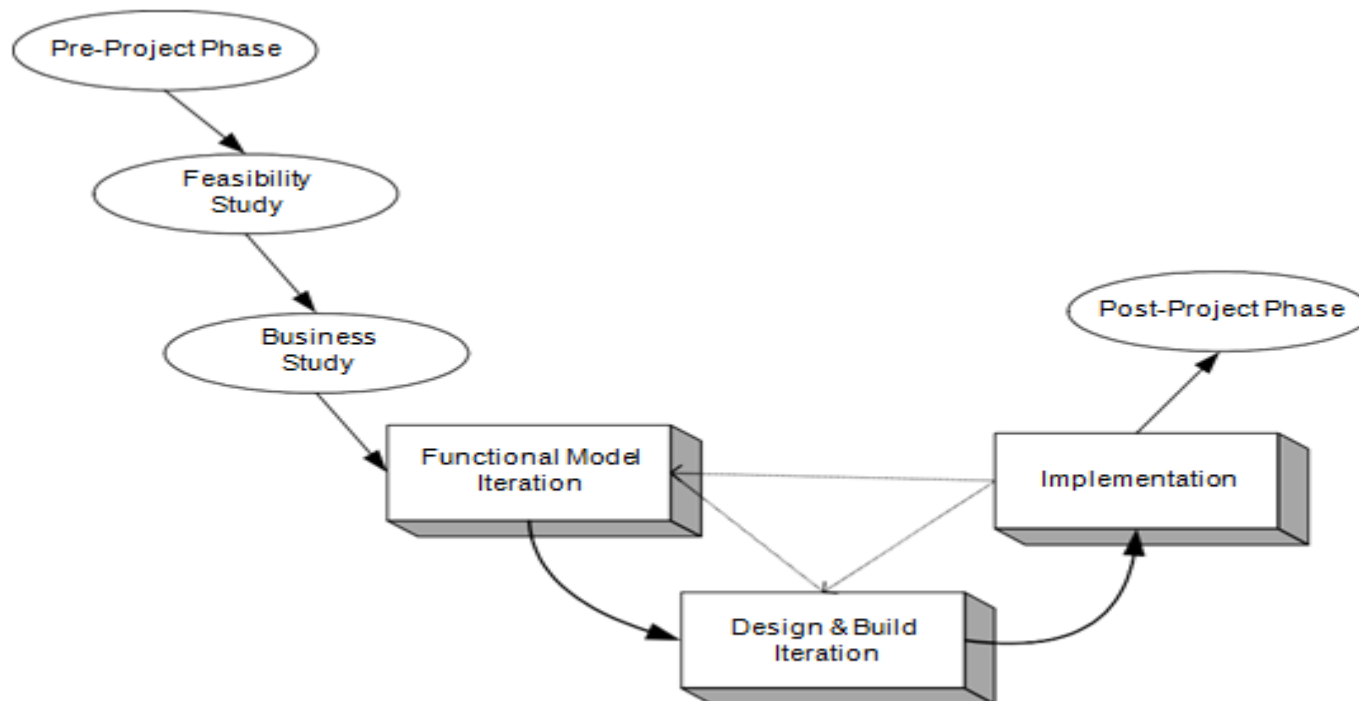


Fig. 2.2 DSDM lifecycle.

The actual DSDM lifecycle is broken down into seven different phases, these are:

1. Pre-Project Phase,
2. Feasibility Study,
3. Business Case Study,
4. the Functional Model Iteration (FMI),
5. the Design and Build Iteration (DBI),
6. the Implementation Phase and
7. the Post-Project Phase.

1. The first three phases (namely, the Pre-Project, Feasibility and Business Studies phases) are done sequentially in order. These phases set the ground rules for the rest of development process allowing users and teams to understand the world within which the application must execute as well as what will be expected of the end product.

2.The Feasibility Study phase is expected only to last a few weeks.

The output of this phase is a “feasibility report” that assesses whether or not to use DSDM for the particular project. It should also consider issues surrounding the people and organizations involved, and define the general scope of the project and its objectives. This phase should also produce an outline plan for the development of the end product.

3.The Business Study phase of the project should have three outputs;

Business Area Definition (BAD),
System Architecture Definition (SAD) and
Outline Prototyping plan:

Business Area Definition. Identifies the high-level requirements and provides a process description of the end product. .

System Architecture Definition. Sketches out the architecture of end system. Note that it is likely that this will evolve over the life of the project.

Outline Prototyping Plan. This states the prototyping strategy to be adopted for the development of the end product.

The 3 core phases of the DSDM are the

- 1.FMI,
2. DBI and
3. Implementation Phase.

1.The *FMI Phase* involves:

Analysis of the features to be designed and implemented. The production of the Functional Model. This is the primary output of this phase. It may include prototype code as well as analysis models. Coding and prototyping. Prototypes may be used to help improve the analysis or understanding of the system. These prototypes may continue to evolve (particularly in the next phase) until the quality level achieved is high enough that they can be used in the delivered system.

2. The *DBI Phase* involves:

Designing and Building the features to be implemented during this phase. This involves reviewing the designs produced so far, the functional prototypes, as well as the creation of code to implement the required functionality. The primary output of this state is the tested system. This system must meet all the requirements selected as essential in the particular iteration being implemented

3. The *Implementation Phase* involves:

The transfer of the completed system from the development environment to the production environment.

The provision of other deliverables such as User training, the creation of the User Manual and the Project Review Report. If issues arise, then the project can be reiterated back to the appropriate phase.

DSDM refines the concept of timeboxing by nesting shorter timeboxes of 2–6 weeks within the overall time frame.

Each time box will typically pass through three phases.

- **Investigation** – a quick pass to see whether the team is taking the right direction.
- **Refinement** – to build on the comments resulting from the review at the end of investigation.
- **Consolidation** – the final part of the timebox to tie up any loose ends.

The **MoSCoW** rules provide the basis on which decisions are made over the entire project, and during any timebox. As timeboxes are fixed, the deliverables from the timebox may vary according to the amount of time left. Essential work must be done – less critical work can be omitted. So, the MoSCoW rules are applied.

MoSCoW stands for:

- **Must** have: fundamental to the project's success “on time”
- **Should** have: important but the project's success does not rely on these **Could** have: can easily be left out without impacting on the project “on budget”
- **Won't** have this time round: can be left out this time and done at a later date.

SCRUM

- SCRUM is a set of rules, procedures, and practices that are all inter- related and that work together to improve the development environment, reduce organizational overheads and ensure that iterative deliverables match the end users requirements.

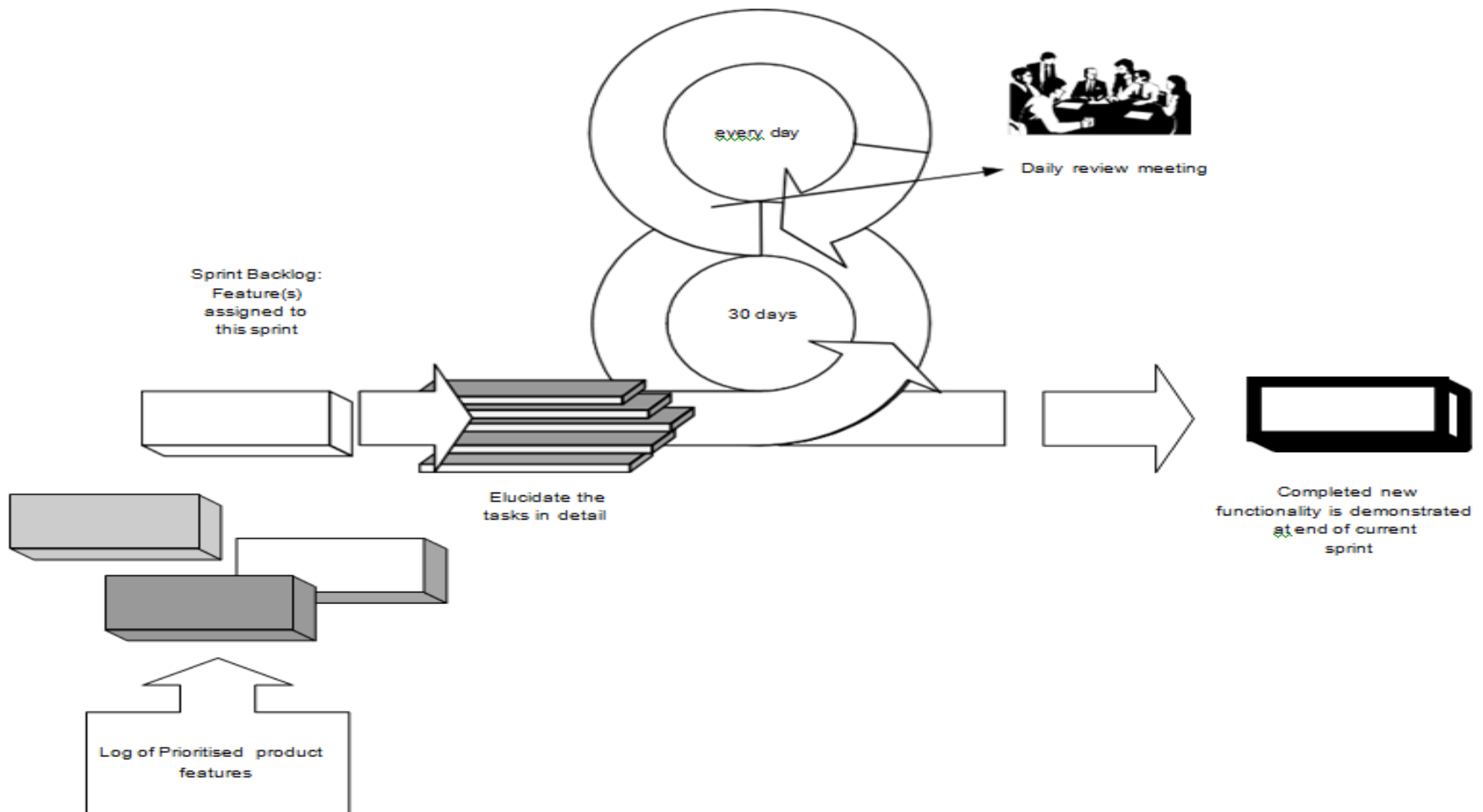


Fig. 2.3 SCRUM.

One of the interesting aspects of SCRUM is that actually it aims to help with the production of a “product” of which software is just one example. The benefits put forward by the proponents of SCRUM are:

- The management and control of development work in an agile manner.
- It explicitly acknowledges that requirements may be changing rapidly within its iterative and incremental approach to product development.
- It is possible to still use existing engineering practices within SCRUM (which may help facilitate the introduction of agile methods into an organization).
- It is an inherently team-based approach and helps to improve communications and co-operation.
- It scales from small projects up to very large projects.
- It helps to identify and then remove any obstacle to the smooth development of the end product

- SCRUM is based on current process control theories and specifically aims to produce the best end result, given the current resources and time available.

as well as having an iterative cycle of only 30 days between iterative deliveries, it also employs daily reviews. These reviews should be short (of, for example, 15-min duration), and should force team members to address the basics of the process, namely they should consider:

- What has been done by team members since the last meeting.
- Is there anything causing a problem? Are there any obstacles to completing their tasks?
- What will each team member do before the next meeting?

Feature-Driven Development

- A *feature* is a schedulable requirement associated with the activity used to realize it. These requirements may be user related requirements (i.e., be able to open a bank account), application behavior requirements (make a backup every 10 min) or internal requirements (provide the ability to turn on debugging for system support). Each feature has a priority and a cost associated with it (for example, an estimate of the amount of person effort required to design, implement and test that feature). Thus, a feature mixes units of requirements with units of management. In addition, features should have the following attributes:
- Features should be small and “useful in the eyes of system stakeholders.”
- Features can be grouped into business-related groupings (called variously
- feature sets or work packages).
- Features focus developers on producing elements within the system that are of tangible benefits to system stakeholders.
- Features are prioritized. Features are schedulable.
- Features have an associated (estimated) cost. Can be grouped together into short iterations (possibly as short as two weeks).

why to consider feature-centric design. The reason for considering yet another method is that iterative lifecycles (such as those promoted by agile methods) tend to be more complex than linear one. They tend to:

- require more planning and re-planning,
- more assessment of where the project has got to go,
- more judgment of what should happen now,
- more monitoring of progress,
- be able to respond more quickly to the current situation (which potentially leads to more planning, etc.).

This involves asking some of the following questions as iterations progress:

- How are we progressing relative to the overall goal of the system?
- What are the priorities now and how have they changed?
- What issues and risks does the project now face?
- How can the issues and risks be addressed or mitigated?

Feature-centric management of agile projects offers this element of control. In FDD, Coad (1999) and Palmer and Felsing (2002) present a five-step process that outlines how a feature-centric approach works, these five processes are:

- Process 1: Develop an overall model of the domain and create an initial feature list.
- Process 2: Build a detailed, prioritized feature list. Process 3: Plan by feature.
- Process 4: Design by feature. Process 5: Build by feature.

We can elaborate on that by considering an iterative feature based lifecycle. This is presented in Figure 2.4. In the diagram next slide, the flow illustrates the following steps:

- **First identify a prioritized feature list.** This can be done by considering the systems' requirements. These can be produced in whatever manner is appropriate. For example, through use cases, a formal requirements specification or user stories. What is required is that they are elaborated sufficiently to allow prioritization and an initial cost estimate to be associated with them.
- **This initial feature list is then used to create a plan of the iterations to be undertaken.** Each iteration should have one or more features associated with it and should not be too long. Each iteration should have a time box associated with it that specifies when it starts and when it finishes.
- **Before each iteration starts, the iteration should be planned in detail.** This involves determining which features are still relevant for that iteration, any revised priorities and an ordering to the features.
- **Once a iteration starts, each iteration is addressed in turn based on their priorities.** At any one time, one or more features will be worked on depending on the size of the feature and the resources available (note that no assumption is made here about how many people will be needed to implement a feature, there could be one developer per feature, two per feature or variable depending upon the step within the feature that is currently being addressed).
- **The iteration stops when the time box finishes.** At the end of the iteration, the current version of the software is tested to ensure it does what it should.
- **If this is the final iteration, then the final version of the system is delivered (if it is not the final iteration,** then the current system may still be delivered to end users for early and frequent feedback). This is possible as each feature should be useful in the eyes of the various project stakeholders in their own right.

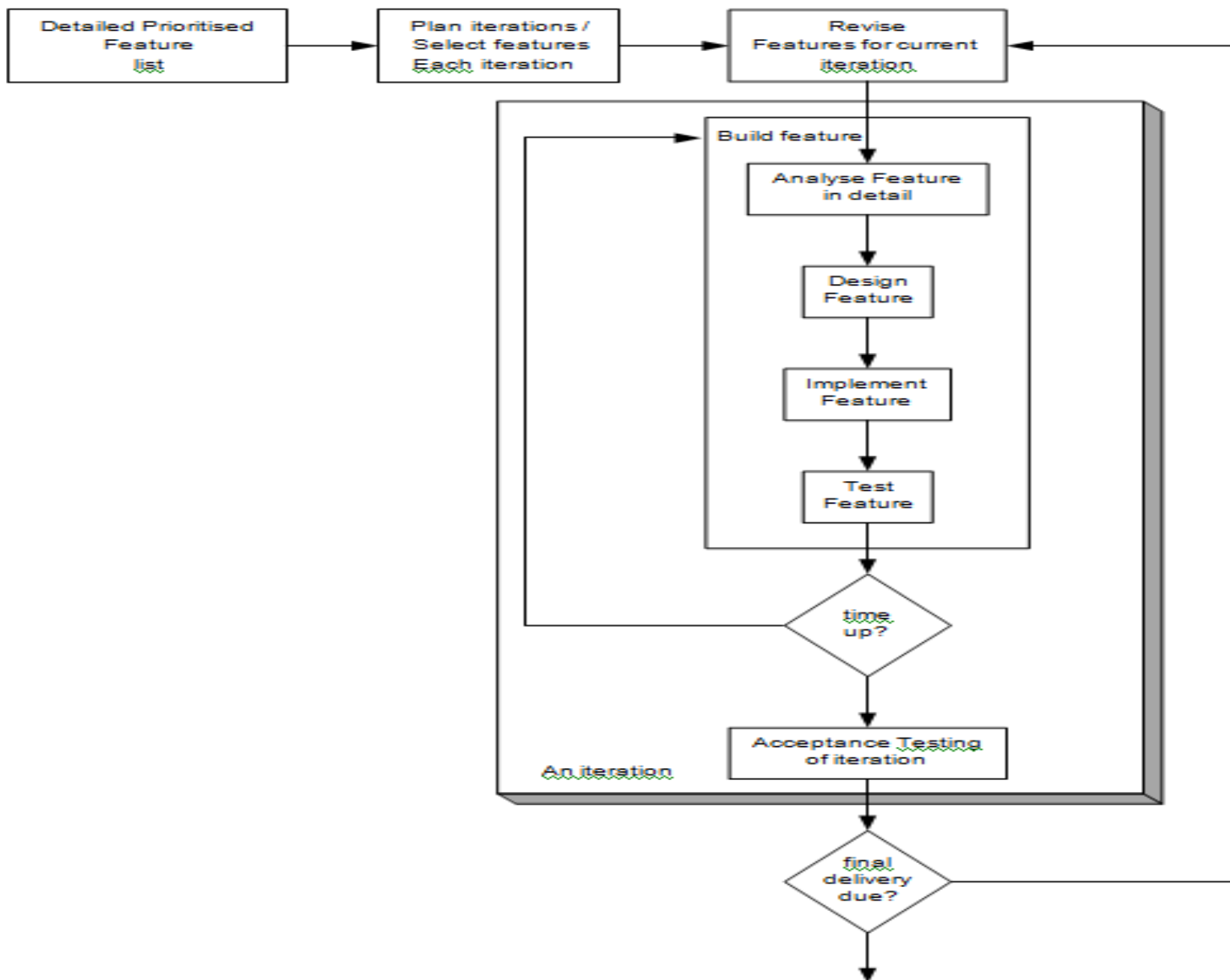


Fig. 2.4 Iterative feature based lifecycle.

Agile Modelling

– Modelling Misconceptions

Before discussing Agile Modelling, it is worth reconsidering some modelling myths and misconceptions that need to be clarified. These have a bearing on Agile Modelling as a modelling misconception can lead to a denial of the benefits that can be accrued from Agile Modeling.

1. Models equal documentation.

Nothing could be further from the truth! A model is *part* of the documentation, but it is by no means sufficient as documentation. That is, a (UML) model, as good as it may be, cannot adequately represent all the information needed to describe how the requirements (functional and nonfunctional), behavioral and structural of a software system are to be implemented.

For example, Figure 3.1 illustrates part of a Rose model developed for a real-world system built on one of the projects I have worked on. This Rose model has links to word documents (such as Overview), screen designs (such as *APS-Frame-View*), design notes (*APS-MVC-Design*), classes (*APSFrameController*, *APS-FrameModel*, *APSFrameView*), Sequence diagrams (View SQM Questions) as well as class diagrams, collaboration diagrams,

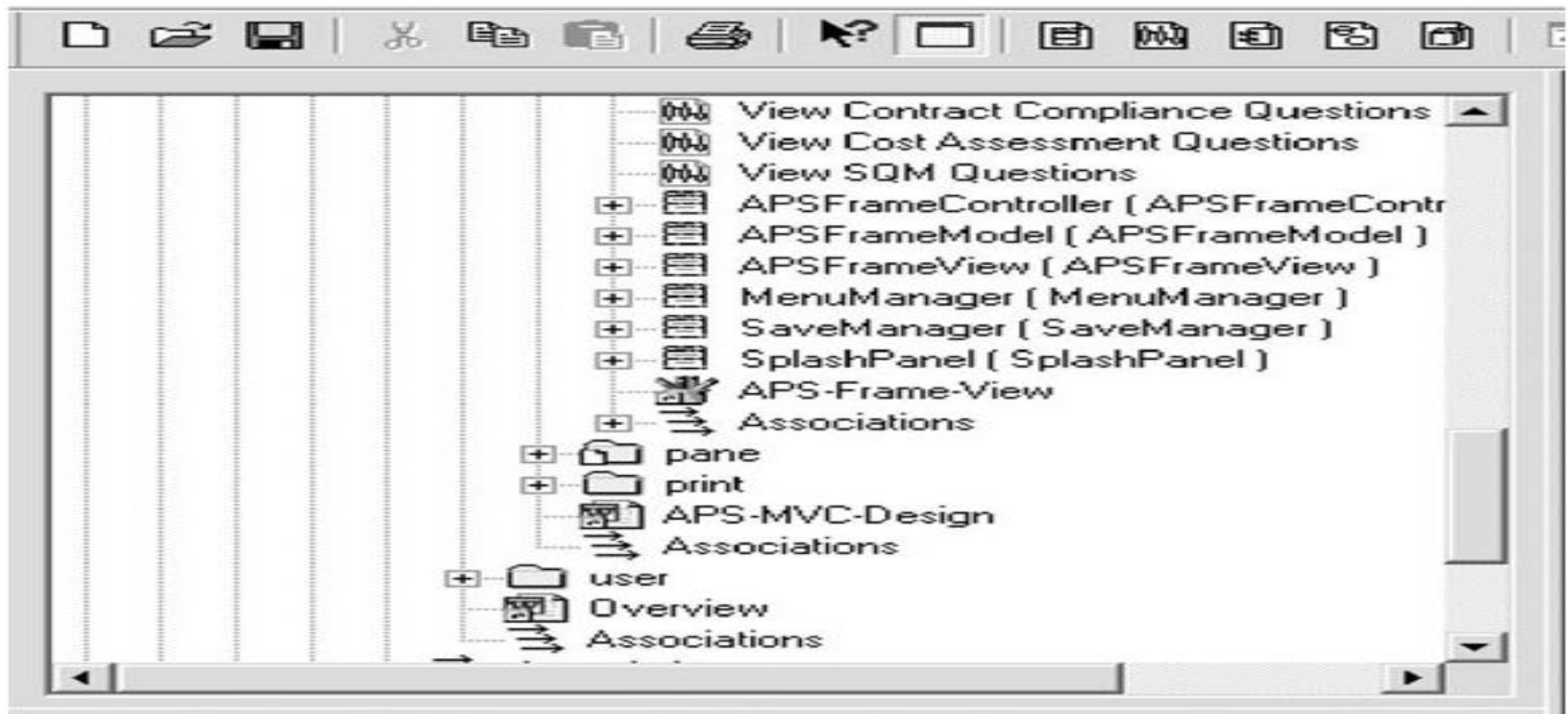


Fig. 3.1 Part of a Rose Model.

2. Modelling implies a heavyweight software process.

Again this is not true. The fact that you are using some form of modelling to describe your system does not mean that you must be using a formal software development process. It may well be that placing the modelling task within the context of a development methodology may well help, but modelling does not equate to a software process.

3. You must “freeze” the requirements.

The point here is that many people believe that you must be able to freeze the requirements before you start to model. In theory this would be great. If you had all the requirements presented to you before you start modelling, then all the questions about what the system should do would be answered at the start. It would also make deciding on what should be in the model easier. However, that is theoretical, in reality requirements change (even in the smallest projects). This can be for a variety of reasons. For example:

- Those who wrote the requirements missed out some details (that may only come to light when the development is progressing);
- The users find their needs change during the lifetime of the project; or The project is a Greenfield software system in which the requirements are difficult to ascertain upfront (or any combination of these).

4. Your design is carved in stone.

- This is a leftover from the more traditional waterfall-based approaches to software development. In an ideal world, you would like to remove as many unknowns and variable elements from software development as possible. Therefore, to have the design “fixed” and never changing would be great. Again this does not work in reality

5. You must use a CASE tool.

- By this I mean that if you are going to do some modelling you must use some form of Computer Aided Software Engineering tool such as TogetherSoft's Together Control Centre or Rational's Rose. Of course, these tools may well make things easier, but they are not mandatory. Indeed in the first edition of Hunt (2003), I used the UML modelling features of the Microsoft tool Visio to generate all the models presented. As a comparison, Figure 3.2 is a diagram in Visio while Figure 3.3 is from Rose and Figure 3.4 from Together. As an extreme, I once worked for a software company, writing Smalltalk software for the financial industry, where they wanted me to use Paint to draw class diagrams. Note all of these (as well as hand-drawn diagrams) represent models.

6. All developers know how to model.

- Generating appropriate, correct, well-formed, understandable models is not trivial. It takes time to get familiar with whatever tools, notation and

7. You can think of everything from the start.

- One problem with UML style models is that they are static – that is, you cannot execute them. So, it is hard to determine whether they cover “enough” or whether you have missed out some critical areas. Therefore, believing that you can think of everything and cover all eventualities at the start of modelling is wrong for all but the simplest systems.

8. Modelling is a waste of time.

- I have both heard, and had to deal with, this myth. This myth represents the extreme opposite of that promoted by the waterfall boys – that is, designing models has no benefit, just get on with the coding – that's what you are delivering after all.

9. The world revolves around the data model.

- That is, the data model describing, for example, the information in the database, is the centre of the universe and the object-oriented model is based on this.

– Agile Modelling

- So what is Agile Modelling? First, it is not a complete methodology in the sense that you can do Agile Modelling and that is sufficient. Rather it is an approach to the modelling aspects of a software development method. That is, it is an add-on to an approach such as the Unified Process (or indeed XP). The Unified Process is actually a framework.

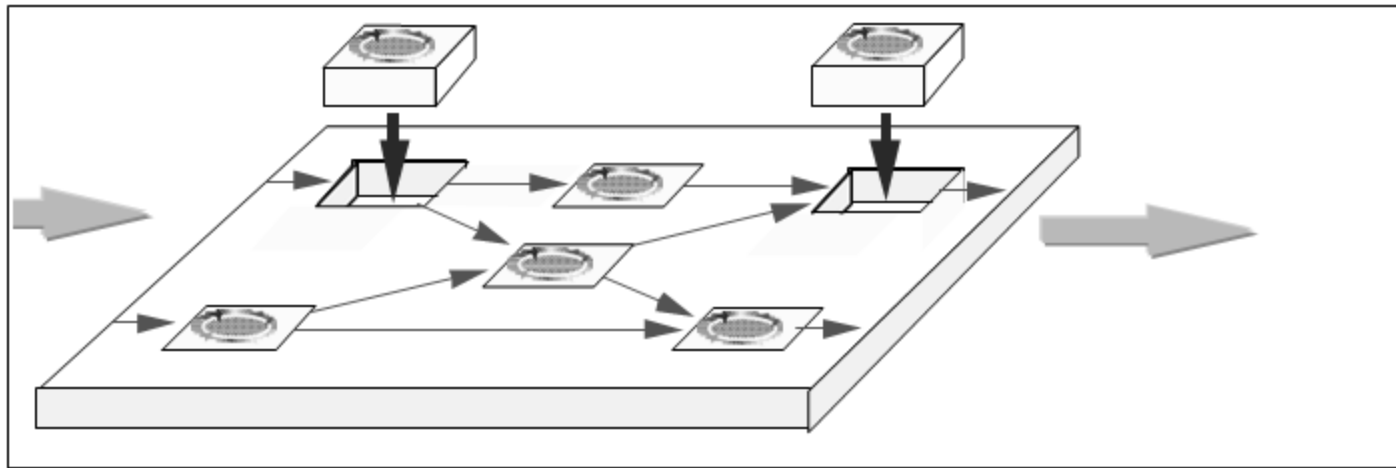


Fig. 3.5 The Unified Process as a framework.

- The use of Agile Modelling is just another technique that can be used to augment the Unified Process. Thus, adding agile modelling to the Unified Process is completely in keeping with the original aims of the Unified Process. This is illustrated in Figure 3.6

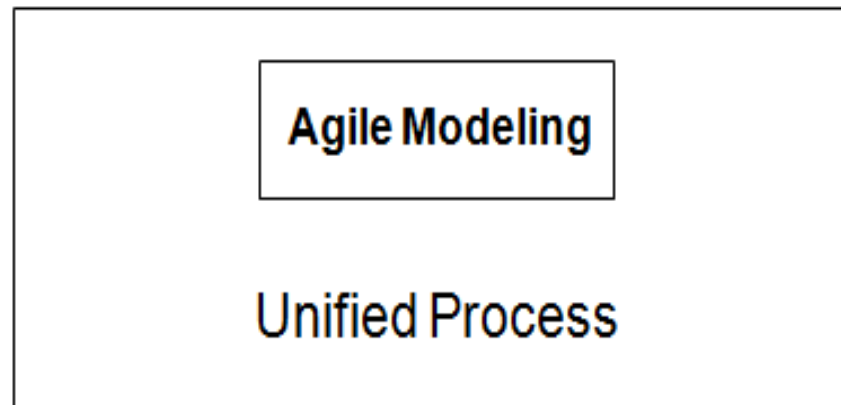


Fig. 3.6 Augmenting the Unified Process with Agile Modelling.

Agile Models. Agile models should:

- *Provide positive value* in that they should have some utility, i.e. someone should need them.
- *Fulfill their purpose* (and no more) so if a model is to be used to clarify how some classes fit together, then that is all that it should clarify.
- *Be understandable* (to their intended audience but not necessarily to every- one).
- *Are sufficiently accurate* for their intended audience! *Are sufficiently consistent* as long as those using the models understand them. *Are sufficiently detailed*. For example, when explaining the structure of a group of classes, the exact set of methods, variables and constructors may not be relevant. *Are as simple as possible*.

Tool Misconceptions

At this point, it is worthwhile considering some misconceptions and myths relating to the use of tools, modeling and UML.

- *UML requires CASE tools.*

This is certainly not true – I can draw a UML diagram freehand on paper, use a simple drawing package such as Paint or indeed with a tool such as Together. It may well be true that to strictly adhere to the UML notation it is easier to use something that knows about UML (and Visio might be such a tool). It may also be true that if you want to generate code from the UML diagrams, using a CASE tool such as Rose or Together makes life easier. But it is not a pre-requisite.

- *Modeling requires the use of CASE tools.*

An extension of the last point is that if you are going to create models as part of your design you need a CASE tool that can manage, manipulate, cross reference, etc., your models. While these features may well be useful, they are not necessary. I can (and indeed have) used simpler tools such as Visio to perform all the modeling necessary for the design of a system. Obviously, the larger the system and the larger the amount of modelling performed, the better a CASE tool may be.

- *Agile modelers don't use CASE tools.*

This is a common misconception by those starting with Agile Modeling. This is partly due to the emphasis of Agile Modeling on using the simplest appropriate tool and if that tool is a white board or a piece of paper, use it. The key word here is *appropriate*. If I need to work something through with one of my colleagues, we might well use a white board or a piece of paper and not worry too much about the accuracy of the UML notation being used. If however, I am trying to describe a complex structure that will need to be referenced by a variety of developers, possibly in multiple locations, then a CASE tool might well be the most appropriate.

- *UML is all you need!*

Some take the view that UML is all you need in terms of notation – if you can't do it in UML (*aka* the design tool you are using), then it is either not relevant to an object-oriented system or not important enough to document. This is not true. There are many aspects of a software project that you may wish to document, but do not fit within the remit of a UML diagram, for example, GUI storyboarding, data modeling, etc.

- *The CASE tool is master.*

This is more a perception than a misconception. It is a perception because users often feel that they are battling with the CASE tool and that they have to work in the way prescribed by the CASE tool. Certainly, I know that some of the colleagues I have worked with over the years have an almost irrational hatred against one well known CASE tool because of the way it forces them to work. Some of this can be overcome with training and some by choosing a suitable tool. The important thing is that the CASE tool should *not* be the master but the servant. It should help you with your work and not hinder it. Thus, finding an appropriate tool (or tools) is important. For example, in one case Together proved to be particularly well suited to the organizations way of working and to the developers experience and background.

Questions on UNIT-1

- What are Agile methods and how do they differ from traditional software development methodologies?
- Name at least three Agile methodologies and highlight their key differences.
- How do Agile methods support iterative and incremental development?
- What are the benefits and challenges of using Agile methods in large-scale projects?
- List and explain the four core values of the Agile Manifesto.
- What are the 12 principles of Agile software development?
- How does the Agile Manifesto promote customer collaboration over contract negotiation?
- Why is Agile Modelling important in software development?
- How does Agile Modelling support just-in-time modelling?
- Define Agile in the context of software development.
- How does Agile respond to changing requirements?
- How does Agile promote team collaboration and self-organization?
- Compare Agile with the Waterfall model.
- What is Extreme Programming (XP)?
- What are the core practices of XP?
- How does XP handle changing requirements?
- What is the role of Test-Driven Development (TDD) in XP?
- What is DSDM and what are its core principles?

- How does DSDM differ from XP and Scrum?
- What are the phases in the DSDM lifecycle?
- What is the importance of user involvement in DSDM?
- What is Scrum and what are its key components?
- Describe the roles in a Scrum team.
- What are the artifacts and ceremonies in Scrum?
- How does Scrum ensure continuous improvement?
- What is Feature-Driven Development (FDD)?
- What are the key activities involved in FDD?
- How does FDD manage large and complex systems?
- Compare FDD with other Agile methodologies like Scrum or XP.
- What are common misconceptions about modelling in Agile?
- Why do some developers believe modelling is unnecessary in Agile?
- How can poor understanding of modelling affect Agile projects?
- What are the best practices of Agile Modelling?
- How does Agile Modelling support "model with a purpose"?
- What are the types of models used in Agile?
- How does Agile Modelling integrate with Agile development practices like Scrum or XP?
- What are some tools that contribute to modelling misconceptions?